A Computer Vision Project Report on

Touchless Interactive Smart Kiosk

using Computer Vision

for Gesture-Driven Touchless Interaction

Submitted to Manipal University Jaipur
Towards the partial fulfillment of the Award of the Degree of
BACHELOR OF TECHNOLOGY
In Computer Science and Engineering
2023–2027

By
Devansh Shukla – 23FE10CSE00741
Aryan Mishra – 23FE10CSE00719

MANIPAL UNIVERSITY JAIPUR

Under the guidance of
Ms. Arshpreet Kaur

Department of Computer Science and Engineering
School of Computer Science and Engineering
Manipal University Jaipur
Jaipur, Rajasthan

1. INTRODUCTION

The proliferation of touch-based interfaces in public environments — such as kiosks, ATMs, and ticketing terminals — has raised growing concerns around hygiene, particularly in the wake of global health crises. Frequent physical contact with shared surfaces facilitates the transmission of pathogens, posing health risks to users. Moreover, conventional touch interfaces present significant accessibility barriers for individuals with motor disabilities or conditions that impair fine motor control.

Touchless human-computer interaction offers a compelling alternative. By leveraging computer vision and gesture recognition, it becomes possible to control a digital interface entirely through natural hand movements, eliminating the need for any physical contact. Such systems are not only hygienic but also more inclusive, reducing dependency on physical dexterity.

This project presents the design and implementation of a Touchless Interactive Smart Kiosk that enables users to navigate a simulated ordering interface using hand gestures captured via a standard webcam. The system detects hand landmarks in real time using MediaPipe, interprets gestures through rule-based logic, and translates them into system-level mouse actions using PyAutoGUI. A web-based kiosk interface developed with Streamlit provides users with a multi-page menu, ordering, and cart management experience — all controlled without physical touch.

2. METHODOLOGY

The system is composed of two loosely coupled modules: a gesture recognition engine and a web-based frontend UI. These modules communicate through a lightweight file-based messaging mechanism, enabling modular development and clean separation of concerns.

2.1 Video Capture and Preprocessing

Live video is acquired from the system's webcam using OpenCV, which provides real-time frame capture at standard resolutions. Each frame is horizontally flipped to create a mirror effect, improving the intuitiveness of cursor control. The frame is then converted from BGR to RGB colour space before being forwarded to the hand detection pipeline, as required by the MediaPipe framework.

2.2 Hand Detection and Landmark Extraction

MediaPipe's Hand solution detects and tracks hand presence within each video frame. Upon detection, the framework returns a set of 21 three-dimensional landmarks corresponding to specific anatomical points on the hand, including fingertips, knuckles, and the wrist. These landmarks are normalized to frame dimensions, providing coordinate values that are resolution-independent. Key landmarks — specifically those corresponding to the index fingertip (Landmark 8), middle fingertip (Landmark 12), thumb tip (Landmark 4), and ring finger base (Landmark 16) — serve as the primary features for gesture interpretation.

2.3 Rule-Based Gesture Recognition

Rather than employing a learned classification model, the system defines gestures using deterministic, rule-based conditions derived from spatial relationships between landmarks. This approach ensures low computational overhead and predictable behaviour. The six supported gestures are as follows:

- Cursor Control: The position of the index fingertip drives the on-screen cursor, with movement mapped linearly from camera space to screen space.
- Left Click: Detected when the Euclidean distance between the thumb tip and index fingertip falls below a defined threshold, simulating a pinch gesture.
- Right Click: Triggered by a pinch between the thumb tip and the middle fingertip.

- Scroll: Activated when both the index and middle fingers are extended, with vertical displacement driving scroll direction and magnitude.
- Swipe Navigation: A positional buffer stores the recent trajectory of the index fingertip. Horizontal displacement exceeding a set threshold within a fixed time window is interpreted as a forward or backward swipe, triggering page navigation.
- Pause Mode: An open-palm gesture, determined by detecting all five fingers in an extended state, suspends gesture processing to allow the user to reposition their hand without unintentional input.

2.4 Coordinate Mapping and Cursor Smoothing

Landmark coordinates from camera space are mapped to screen coordinates through linear interpolation, accounting for the active input region of the frame. To mitigate involuntary cursor jitter — a common artefact of real-time hand tracking — a weighted moving average is applied to successive cursor positions. This smoothing filter blends the current estimated position with prior positions, yielding fluid and controllable cursor movement without perceptible lag.

2.5 System Integration and Communication

The gesture recognition engine and the Streamlit-based frontend operate as independent processes. Inter-module communication is achieved via a shared plain-text file, gesture_command.txt, to which the gesture engine writes recognized commands (e.g., click, scroll_up, swipe_left). The frontend periodically polls this file and updates its session state accordingly. This file-based protocol avoids the complexity of inter-process socket communication while maintaining adequate responsiveness for a kiosk application.

2.6 Frontend Interface

The kiosk UI is built with Streamlit and organized across four pages: Home, Menu, Order, and Help. The Menu page presents selectable items associated with prices. The Order page maintains a cart, dynamically computing totals based on user selections. Session state management within Streamlit persists cart data across page transitions. All interactions — item selection, page navigation, and cart management — are driven exclusively by gesture commands received from the recognition engine.

3. RESULTS

The implemented system successfully demonstrates a functional touchless kiosk experience. All six core gestures were recognized and executed reliably under controlled lighting conditions. Cursor control using the index finger was smooth and responsive, with the weighted smoothing filter effectively reducing jitter to a level suitable for practical interaction.

Navigation across the four UI pages operated as intended, with swipe gestures correctly triggering forward and backward page transitions. The cart system accurately reflected item additions and removals initiated through gesture-based clicks, and real-time total calculation was consistently correct. The pause mode proved particularly valuable in preventing accidental inputs during hand repositioning.

From a performance standpoint, the system operated at a frame rate adequate for interactive use on a standard laptop. The use of rule-based gesture recognition, rather than a deep learning classifier, contributed to consistently low inference latency, as no model inference was required beyond MediaPipe's internal hand detection pipeline.

Several limitations were identified during evaluation. Gesture accuracy was sensitive to ambient lighting; insufficient or inconsistent illumination reduced the reliability of landmark detection. Pinch distance thresholds required careful calibration and may not generalize uniformly across users with different hand geometries. The swipe gesture was occasionally triggered unintentionally during cursor repositioning. Future work could address these limitations through adaptive thresholding, per-user calibration, and temporal gesture state machines for improved robustness.

4. CONCLUSION

This project demonstrates that a practical, touchless kiosk interface can be realized using accessible tools — a standard webcam, MediaPipe, OpenCV, PyAutoGUI, and Streamlit — without resorting to complex machine learning pipelines. The rule-based gesture engine offers transparency, predictability, and computational efficiency, making the system suitable for deployment in resource-constrained environments. The project affirms the viability of computer vision as a foundation for hygienic, accessible human-computer interaction in public-facing applications.

5. REFERENCES

[1] Google LLC. (2023). MediaPipe Hands: On-Device Real-Time Hand Tracking. https://google.github.io/mediapipe/solutions/hands

[2] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. https://opencv.org/documentation/

[3] Agrawal, A. (2021). PyAutoGUI Documentation. https://pyautogui.readthedocs.io/en/latest/

[4] Streamlit Inc. (2023). Streamlit Documentation. https://docs.streamlit.io/

[5] Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2nd ed.). Springer. https://szeliski.org/Book/

[6] Zhang, F., et al. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214.